

Aula Completa

SQLite no React Native com Expo

Objetivos da Aula

Ao final desta aula, o aluno será capaz de:

- Entender quando usar **SQLite** no mobile e quando não usar
- Configurar corretamente o `expo-sqlite` no Expo
- Aplicar CRUD com SQL parametrizado e repositório
- Usar **constraints, índices, transações e migrações**
- Modelar dados para apps offline-first em React Native

Por que SQLite no Mobile?

Nem todo app precisa de internet o tempo todo.

Com SQLite, você consegue:

- Salvar dados localmente no dispositivo
- Continuar funcionando **offline**
- Sincronizar depois com backend (quando houver internet)
- Melhorar a experiência do usuário (menos loading e menos consumo de rede)

Quando usar SQLite (e quando não)

Use SQLite quando você tiver:

- Dados estruturados (listas, filtros, relatórios)
- Relacionamentos entre entidades
- Necessidade de consulta rápida local
- Aplicativo com modo offline

Evite SQLite quando:

- Só precisa salvar 2 ou 3 preferências simples
- O dado é efêmero e pode ser perdido sem problema

O que é SQLite?

SQLite é um banco relacional leve, embarcado no próprio app.

- Não precisa instalar servidor
- Funciona por arquivo local (ex: `app.db`)
- Usa SQL padrão (`CREATE` , `INSERT` , `SELECT` , etc.)
- Muito usado em apps mobile (Android e iOS)



Conceitos que importam no SQLite

- **Transação:** bloco atômico de operações (ou tudo salva, ou nada salva)
- **Constraint:** regra de integridade (ex: `NOT NULL`, `UNIQUE`)
- **Índice:** acelera buscas e ordenações
- **Foreign Key:** vincula tabelas relacionadas
- **Migração:** evolução controlada do schema ao longo do tempo

AsyncStorage vs SQLite

Recurso	AsyncStorage	SQLite
Estrutura	Chave/valor	Tabelas relacionais
Consultas complexas	Limitado	Excelente
Volume de dados	Pequeno/médio	Médio/grande
Filtros/ordenação	Manual em JS	SQL nativo
Integridade	Baixa	Alta (constraints)

Regra prática:

- Preferir **AsyncStorage** para configurações simples.
- Preferir **SQLite** para dados estruturados e consultas.

Arquitetura da Persistência Local

Fluxo simplificado:

```
[Interface React Native]
  -> [Camada de serviço DB]
    -> [expo-sqlite]
      -> [Arquivo local SQLite]
```

Separar UI da camada de banco facilita manutenção, reuso e testes.

Instalação

```
npx expo install expo-sqlite
```

Para o projeto completo de galeria com mapa (trabalho), também usaremos:

```
npx expo install expo-image-picker expo-location react-native-maps
```

Modos de uso no expo-sqlite

Você pode usar APIs síncronas ou assíncronas.

- `openDatabaseSync` / `runSync` / `getAllSync` : simples e direto
- `openDatabaseAsync` / `runAsync` / `getAllAsync` : melhor para fluxos não bloqueantes

Em telas com muito processamento, prefira assíncrono para evitar travamentos perceptíveis.



Estrutura sugerida

```
src/  
  db/  
    database.ts  
  repositories/  
    photosRepository.ts  
    migrationsRepository.ts  
  screens/  
    HomeScreen.tsx  
    MapScreen.tsx
```

Conexão e inicialização do banco

Arquivo: `src/db/database.ts`

```
import * as SQLite from "expo-sqlite";

export const db = SQLite.openDatabaseSync("galeria.db");

export function initDatabase() {
  // Melhora concorrência entre leitura e escrita
  db.execSync(`
    PRAGMA journal_mode = WAL;
    PRAGMA foreign_keys = ON;

    CREATE TABLE IF NOT EXISTS photos (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      title TEXT NOT NULL,
      image_uri TEXT NOT NULL,
      latitude REAL,
      longitude REAL,
      created_at TEXT NOT NULL
    );

    CREATE INDEX IF NOT EXISTS idx_photos_created_at
    ON photos (created_at DESC);
  `);
}
```



Explicando a tabela photos

Campo	Tipo	Descrição
id	INTEGER	Identificador único
title	TEXT	Título da foto
image_uri	TEXT	Caminho local da imagem
latitude	REAL	Latitude da captura
longitude	REAL	Longitude da captura
created_at	TEXT	Data/hora em ISO

`created_at` em ISO (`new Date().toISOString()`) facilita ordenação e comparação temporal.

+ Inserindo dados

Arquivo: `src/repositories/photosRepository.ts`

```

import { db } from "../db/database";

type NewPhotoInput = {
  title: string;
  imageUri: string;
  latitude: number | null;
  longitude: number | null;
};

export function insertPhoto(input: NewPhotoInput) {
  const stmt = db.prepareSync(`
    INSERT INTO photos (title, image_uri, latitude, longitude, created_at)
    VALUES ($title, $image_uri, $latitude, $longitude, $created_at)
  `);

  try {
    stmt.executeSync({
      $title: input.title,
      $image_uri: input.imageUri,
      $latitude: input.latitude,
      $longitude: input.longitude,
      $created_at: new Date().toISOString(),
    });
  } finally {
    stmt.finalizeSync();
  }
}

```

Parametrização (`$title` , `$image_uri`) evita SQL injection e erro de escape.

Listando dados

```
export function listPhotos() {  
  return db.getAllSync<{  
    id: number;  
    title: string;  
    image_uri: string;  
    latitude: number | null;  
    longitude: number | null;  
    created_at: string;  
  }>(`  
    SELECT id, title, image_uri, latitude, longitude, created_at  
    FROM photos  
    ORDER BY created_at DESC  
  `);  
}
```

Consultas com filtro e paginação

```
export function searchPhotosByTitle(term: string, limit = 20, offset = 0) {  
  return db.getAllSync(  
    `SELECT id, title, image_uri, latitude, longitude, created_at  
    FROM photos  
    WHERE title LIKE ?  
    ORDER BY created_at DESC  
    LIMIT ? OFFSET ?`  
    ,  
    [`%${term}%`, limit, offset],  
  );  
}
```

Esse padrão evita carregar a tabela inteira de uma vez.

Atualizando e removendo

```
export function updatePhotoTitle(id: number, title: string) {
  db.runSync(
    `
      UPDATE photos
      SET title = ?
      WHERE id = ?
    `,
    [title, id],
  );
}

export function deletePhoto(id: number) {
  db.runSync(
    `
      DELETE FROM photos
      WHERE id = ?
    `,
    [id],
  );
}
```

Transação na prática

Se você precisar inserir em mais de uma tabela, use transação:

```
export function savePhotoWithLog(title: string, imageUrl: string) {
  db.execSync("BEGIN");

  try {
    db.runSync(
      "INSERT INTO photos (title, image_uri, created_at) VALUES (?, ?, ?)",
      [title, imageUrl, new Date().toISOString()],
    );

    db.runSync("INSERT INTO logs (event, created_at) VALUES (?, ?)", [
      "PHOTO_CREATED",
      new Date().toISOString(),
    ]);

    db.execSync("COMMIT");
  } catch (error) {
    db.execSync("ROLLBACK");
    throw error;
  }
}
```

Relacionamentos com Foreign Keys

Exemplo de duas tabelas: álbuns e fotos.

```
CREATE TABLE IF NOT EXISTS albums (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL UNIQUE  
);  
  
CREATE TABLE IF NOT EXISTS photos (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  album_id INTEGER,  
  title TEXT NOT NULL,  
  image_uri TEXT NOT NULL,  
  created_at TEXT NOT NULL,  
  FOREIGN KEY (album_id) REFERENCES albums(id) ON DELETE SET NULL  
);
```

Isso aumenta consistência do domínio de dados.

Migrações de schema

Conforme o app evolui, o banco também evolui.

Padrão comum:

1. Ler `PRAGMA user_version`
2. Aplicar scripts pendentes (`v1 -> v2 -> v3`)
3. Atualizar `user_version`


```

export function runMigrations() {
  const current =
    db.getFirstSync<{ user_version: number }>("PRAGMA user_version;")
    ?.user_version ?? 0;

  if (current < 1) {
    db.execSync(`
      CREATE TABLE IF NOT EXISTS photos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        image_uri TEXT NOT NULL,
        latitude REAL,
        longitude REAL,
        created_at TEXT NOT NULL
      );
      PRAGMA user_version = 1;
    `);
  }

  if (current < 2) {
    db.execSync(`
      ALTER TABLE photos ADD COLUMN synced INTEGER DEFAULT 0;
      PRAGMA user_version = 2;
    `);
  }
}

```

Exemplo de uso na tela

```
import { useEffect, useState } from "react";
import { initDatabase } from "../src/db/database";
import { listPhotos } from "../src/repositories/photosRepository";

export default function App() {
  const [photos, setPhotos] = useState<any[]>([]);

  useEffect(() => {
    initDatabase();
    setPhotos(listPhotos());
  }, []);

  return null;
}
```

Tratamento de erros e validações

Boas práticas no repositório:

- Validar entrada antes de inserir (`title` vazio, URI inválida)
- Encapsular erros técnicos com mensagens de domínio
- Nunca deixar erro silencioso no banco

Exemplo rápido:

```
if (!input.title.trim()) {  
    throw new Error("Título obrigatório.");  
}
```



Performance: o que mais impacta

- Criar índices para campos de busca/ordenação
- Evitar `SELECT *` quando não precisa de todas colunas
- Pagar listas grandes (`LIMIT/OFFSET`)
- Evitar loop com muitas queries pequenas

Regra de ouro: meça os gargalos antes de otimizar.

Offline-first e sincronização

Padrão comum em apps reais:

1. Salva no SQLite imediatamente
2. Marca registro com `synced = 0`
3. Quando houver internet, envia para API
4. Marca `synced = 1` após sucesso

Isso melhora UX e reduz perda de dados em conexão instável.

Boas práticas com SQLite

- Sempre criar tabela com `IF NOT EXISTS`
- Evitar SQL montado por concatenação de strings
- Usar parâmetros (`?` ou `$param`) para segurança
- Criar funções de repositório para centralizar regras
- Usar transações para operações críticas
- Versionar schema com migrações
- Tratar cenários offline e falhas de permissão



Atividade rápida em sala

Objetivo: validar CRUD completo.

1. Criar tabela `notes` (`id` , `text` , `created_at`)
2. Inserir 3 registros
3. Listar e mostrar na tela
4. Editar 1 registro
5. Excluir 1 registro
6. Recarregar app e confirmar persistência
7. Criar índice para `created_at`
8. Criar migração adicionando coluna `pinned`

Ponte para o Projeto Final

Na próxima atividade (trabalho):

- Usuário escolhe/tira foto
- App pega localização
- App salva no SQLite (`image_uri` , `latitude` , `longitude`)
- Galeria lista fotos salvas
- Mapa mostra marcadores de cada foto
- Registros podem ser sincronizados depois com backend

Tudo funcionando mesmo sem internet.

Conclusão

Hoje você aprendeu:

- O papel do **SQLite** no desenvolvimento mobile
- Como criar e usar um banco local com `expo-sqlite`
- Como estruturar CRUD, transações, índices e migrações
- Como preparar base técnica para uma galeria georreferenciada offline-first

Pronto para construir apps offline-first de verdade.